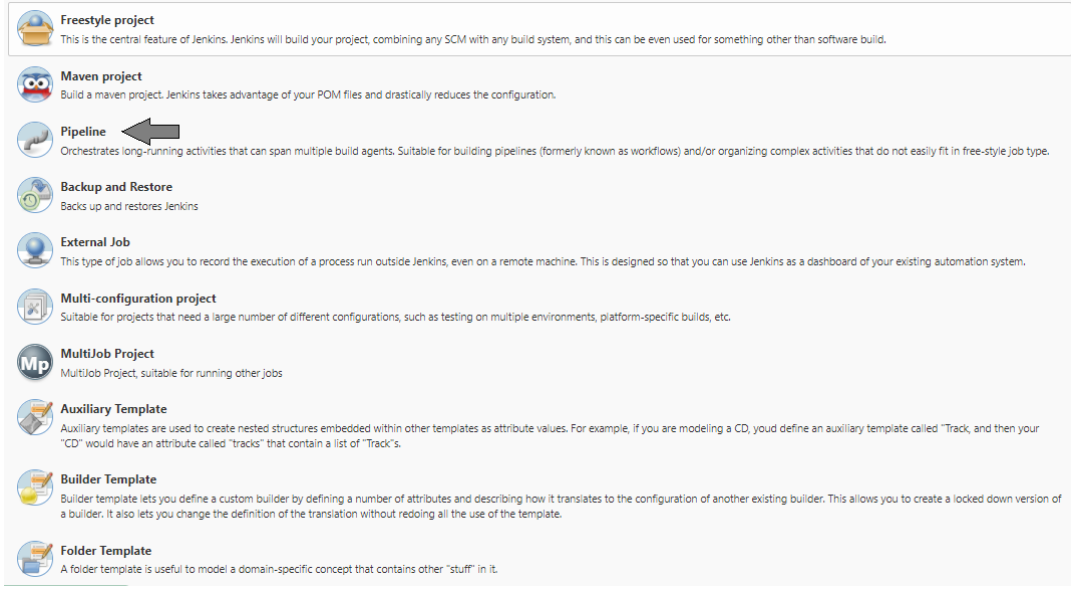
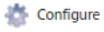
Parameterizations & Configurations

Created by Johnson, Maurice, last modified yesterday at 7:43 PM

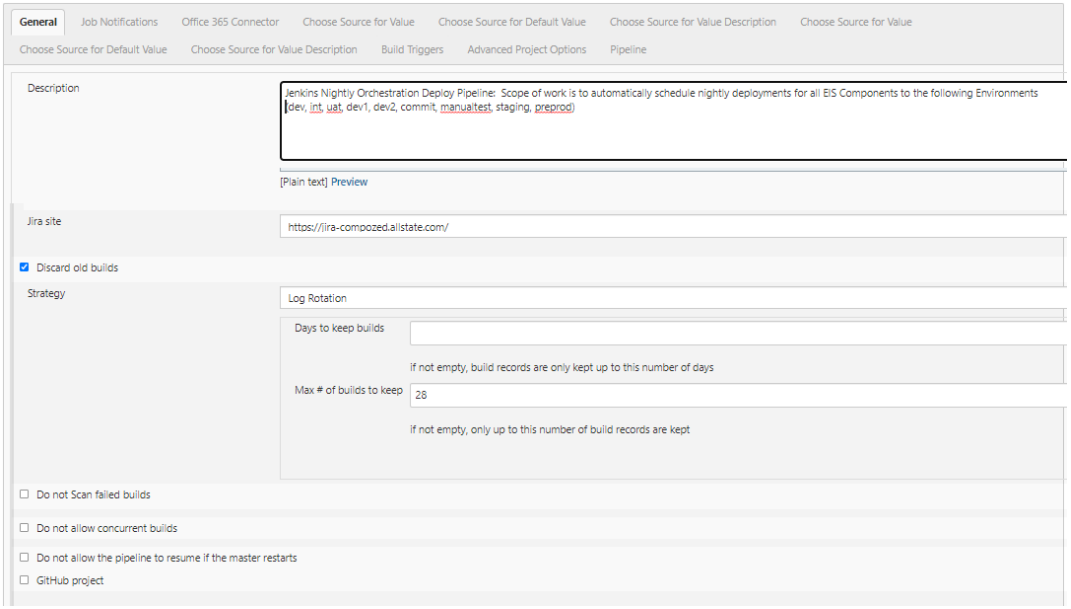
What is Parameterization? You might find yourself asking. The CloudBees Jenkins Platform allows for a number of different Pipelines as well as a number of configurations for those pipelines. "Parameterization" of course being one of them. As you can imagine, with **37** Jenkins Pipelines, in addition to the Orchestration Pipeline the level of communication grows exponentially with each additional pipeline. To leverage the mathematical equation, "**$y = a (1+ r)^x$**" each pipeline has the capability of such an Exponential Growth Rate compounded by "**X**" number of pipelines. So ok, Maurice, I get it, there is a lot to consider, **So how did we tie it together?**

We accomplished this feat by Standing up one of several Jenkins Workflow Types, "**Pipeline**" and added "**Parameterization"** Types within the "**Office 365 Connector**".



Before we can begin however, lets start at the beginning. Navigating to the Orchestration Pipeline, the reader will observe a "Configure" Gear 

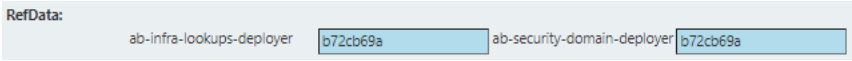
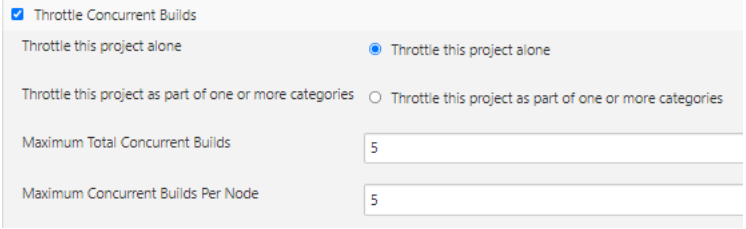
. Proceed to click on this gear. The Jenkins Portal Page will refresh and the reader is presented with the Orchestration Pipeline configurations and parameterizations. As you can see there are several tabs: (1) General (2) Job Notifications (3) Office 365 Connector (4) Build Triggers (5) Advanced Project Options and (6) Pipeline. The "**Office 365 Connector**" Tab is where most of the magic happens, but lets not skip anything and start at the beginning. After our Screen shot, the Configuration and Parameterizations have been broken out into a "**Configurations & Parameterizations Reference Table**" below. It was felt in this way that the reader might gain the most success from reading this material and understanding the overall effort of standing up this pipeline.



Configurations & Parameterizations Reference Table

General Tab			Comments	Value
	Description		Allows the engineer to provide clarity to the scope or purpose of the pipeline	Jenkins Nightly Orchestration Deploy Pipeline: Scope of work is to automatically schedule nightly deployments for all EIS Components to the following Environments (dev, int, uat, dev1, dev2, commit, manualtest, staging, preprod)
	Jira site		This is the default value for pipelines and is rarely altered	https://jira-compozed.allstate.com/
	Discard old builds		Allows the engineer to means of controlling how many builds and or deployments will remain within the overall history of the pipeline.	
		Strategy		Log Rotation
		Days to keep builds		
		Max # of builds to keep		28
Job Notifications			Although not used by this project, this provides the means of triggering events that would deliver an email if the build and or deployment failed. Or maybe it was completed successfully. There are other Communication Protocols offered, like "Slack" Messaging. It might be worth looking into someday, but for now, we did not leverage it.	
Office 365 Connector Tab			As suggested before this is where the magic happens. All Parameterization resides in this section. Referencing the Screen Shot above you will notice there are **Five(5)** additional sections between Office 362 Connector Tab & Build Triggers Connector Tab. They are as follows: (1) Extended Choice Parameter (2) Active Choices Reactive Reference Parameter (3) Active Choices Reactive Reference Parameter (4) Extended Choice Parameter (5) Active Choices Reactive Reference Parameter	
	This project is parameterized		This Boolean Value Checkbox indicates that the pipeline is parameterized.	Yes
		Extended	This represents the **Docker Image Artifactory Choices**	**Name**: dockerRepository

		Choice Parameter	This will in essence create a Parameterized Dropdown with 2 indices, the first containing the default value. The use of the delimiter, ',', will convert the dropdown to a flattened string, with the comma ',' separating each indices. When the value of the dropdown is retrieved it can be readily parsed and converted to a String Array. - ***To note, this will be true for all parameterization and the retrieval of their associative selected values and or input values.***	**Basic Parameter Types**: Yes **Parameter Type**: Single Select **Number of Visible Items**: 2 **Delimiter**: , **Chose Source for Value - Value**: eis-central-remote.artifactory.allstate.com, abeis.artifactory.allstate.com **Chose Source for Value - Default Value:** eis-central-remote.artifactory.allstate.com
		Active Choices Reactive Reference Parameter	This represents the **Kubernetes Namespace Choices** This will create a Parameterized Radio Button Group with 8 total single select options, the default value being Kubernetes Namespace "**dev1**". To accomplish this Formatted HTML was injected as a Groovy Script. Reference the Code Block below; **Groovy Script - Formatted HTML** Expand source	**Name**: allNameSpaces **Script/Groovy Script**: Yes **Groovy Script**: Reference the Code Block **use Groovy Sandbox**: Yes **Choice Type**: Formatted HTML
		Active Choices Reactive Reference Parameter	This represents the **EIS Module/Core Dependency-Configuration Component Choices** This will create a Parameterized Series of of Multi-Select CheckBoxes. There are a total of 2 Multi-Select Boxes. All components are selected by default, with the exception of "Core-Dependency" To accomplish this Formatted HTML was injected as a Groovy Script. Reference the Code Block below; **Groovy Script - Formatted HTML** Expand source	**Name**: eisCoreDependency **Script/Groovy Script**: Yes **Groovy Script**: Reference the Code Block **use Groovy Sandbox**: Yes **Choice Type**: Formatted HTML
		Active Choices Reactive Reference Parameter	This represents the **EIS Module/Static Reference Component Choices** This will create a Parameterized Series of of Multi-Select CheckBoxes. There are a total of 5 Multi-Select Boxes. All components are selected by default, with the exception of "Core-Dependency" To accomplish this Formatted HTML was injected as a Groovy Script. Reference the Code Block below; Expand source	**Name**: eisStaticReference **Script/Groovy Script**: Yes **Groovy Script**: Reference the Code Block **use Groovy Sandbox**: Yes **Choice Type**: Formatted HTML
		Active Choices Reactive Reference Parameter	This represents the **EIS Module/Base Component Choices** This will create a Parameterized Series of of Multi-Select CheckBoxes. There are a total of 10 Multi-Select Boxes. All components are selected by default, with the exception of "Core-Dependency" To accomplish this Formatted HTML was injected as a Groovy Script. Reference the Code Block below; Expand source	**Name**: eisBase **Script/Groovy Script**: Yes **Groovy Script**: Reference the Code Block **use Groovy Sandbox**: Yes **Choice Type**: Formatted HTML
		Active Choices Reactive Reference Parameter	This represents the **EIS Module/Engage Sleep Time** A single select CheckBox, whose default is not Checked, will add a Sleep time in between the Docker Image Deployments of Deployers and their designated APPs. I would like to express caution here, because it will add 79 minutes to the overall Deployment Time. Those Sleep Times are as follows: 1. Core Dependency - 2 Minutes 2. Static Reference a. Deployers 8 Minutes b. APPs 3 Minutes 3. Base a. Deployers 2 Minutes b. APPs 0 Minutes Expand source	**Name**: engageSleep **Description**: ***Engage Sleep Times Between Deployers & Associative APPs. CAUTION this adds 79mins - (StaticRef Comps: 8min Deployers & 3min APPs, Base Comps: 2min Deployers & 0min APPs)*** **Basic Parameter Type**: Yes **Parameter Type**: Textbox **Number of Visible Items**: 1 **Delimiter**: ,
		Extended Choice Parameter	This represents the **EIS Module/Component Docker Image Tag Override Value** This will create a single, out of the box, Textbox, whose value will **supersedes** any **Docker Image Tag Value** entered within the next section "**imageTags**". As the name implies, it is to provide an Override so that the entire Suite of Application Components may be installed at the same identical base Docker Image Tag Version.	**Name**: imageTagOverride **Description**: ***To Provide a Value Here will Override all Subsequent Docker Image Tag Values that are referenced below in the next section*** **Basic Parameter Type**: Yes **Parameter Type**: Textbox **Number of Visible Items**: 1 **Delimiter**: ,
		Active	This represents the **EIS Docker Image Tag Input Values**	**Name**: imageTags

		Choices Reactive Reference Parameter	This will create a Parameterized Series of individual Single Entry Textboxes each catered towards the individual EIS Component. There are a total of 37 Single Entry Textboxes. To accomplish this Formatted HTML was injected as a Groovy Script. It is worth noting here, when looking at the Orchestration Pipeline Deployment Interface , some of the Textboxes have a **Light Blue Background Color**. This is to signify that the Vender EIS has provided a Customized Docker Image that is unique to Allstate's Requirements/needs. As the project matures, it is foreseen that more customization will take place.  Code Block follows **Groovy Script - Formatted HTML** Expand source	**Script/Groovy Script**: Yes **Groovy Script**: Reference the Code Block **use Groovy Sandbox**: Yes **Description**: Provides the Jenkins Operator/Systems Engineer/DEVOps Engineer the means of manually entering the necessary pipeline configurations for this job. **Choice Type**: Formatted HTML
	Throttle Concurrent Builds		Another new feature, which has yet to be fully vetted, is to run the EIS Base Modules/Components in parallel. By doing so, throttling concurrency would require configuration so that we do not overload the Jenkins Node Build Servers. Reference the screen shot below.  	**Throttle Concurrent Builds**: Yes **Throttle this project alone**: Yes **Maximum Total Concurrent Builds**: 5 **Maximum Concurrent Builds per Node**: 5
Build Triggers Tab			The Build Triggers section is just that, it allows the DEVOps and or Systems Engineer to configure a deployment based upon several or singular events. Predominately at Allstate, we will "GitHub hook trigger for GITScm polling" which indicates a polling mechanism against a GiT Repository Main Branch will be in place. This means when a Merge into the Main GiT Repository Branch takes place, polling will trigger the Jenkins Pipeline to Kick Off. For our project however, as of now, we are manually running our Jenkins Deployments. Longterm however we have been asked to schedule them to run either Nightly or Early Morning Eastern Standard Time. In order to do this all the DEVOps and or Systems Engineer would need to do is check the "**Build Periodically**" Build Trigger and provide a schedule. The Jenkins scheduler is very reminiscent to the Unix/Linux Cron Scheduler and has maintained the same scheduling format. Lets say for example we would like our Deployments to KickOff between the hours of 6:00am and 8:00am. The Cron Schedule would be the following; "**0 6-8** * * *" 	
Advanced Project Options Tab			Presently not used.	
Pipeline			The Pipeline Section is the other part of the magic so to speak. As mentioned earlier within the Orchestration Pipeline Introductory our project is leveraging jenkinsFile.groovy Files to act as a "**BootStrap**" for the Jenkins Pipeline Deployments. These jenkinsFile.groovy Files are stored within a Source Code Management(SCM), in our case the GiT Repository. As can be seen from the screen shot below, the Orchestration Pipeline Jenkins File is named **"nightly-orchestration.groovy".** To check out the Groovy File either navigate to the GiT Repository for the latest copy. Or you can check out the code block below; (Current as of this date 12 Aug 2021)	**Pipeline script from SCM** **SCM**: Git **Repository**: https://github.allstate.com/AB-EIS/ab-eis-pas **Credentials**: sys-abeis/***** **Branches**: */master **Script Path:** jenkins/nightly-orchestration.groovy **Lightweight checkout:** Yes

Pipeline script from SCM

SCM: Git

Repositories

Repository URL: https://github.allstate.com/AB-EIS/ab-eis-pas

Credentials: sys-abeis/******

Branches to build

Branch Specifier (blank for 'any'): */master

Repository browser: (Auto)

Additional Behaviours: Add

Script Path: jenkins/nightly-orchestration.groovy

Lightweight checkout ☑

Pipeline Syntax

Expand source

Ok, now that was a significant amount of configurations and parameterizations. Now that we know "**How**" the Orchestration Pipeline was created whats next? **How do we Operate the pipeline**? That is a very good question and will be answered in the next section of the User Manual.

No labels